A VOCATIONAL TRAINING PROJECT REPORT

YĀTRĀ AI: DYNAMIC TRAVEL PLANNER

An Intelligent Geospatial Recommendation System for Offbeat Indian Tourism

Candidate Name:	[Enter Your Name Here]
Enrollment / Roll No:	[Enter Enrollment Number Here]
Course & Stream:	Vocational Training Program in IT & Software Development
Host Organization:	Department of Information Technology
Project Mentor:	[Enter Mentor Name Here]

CHAPTER 1: INTRODUCTION & LITERATURE SURVEY

1.1 Project Background

Tourism in India is one of the largest contributors to the national economy and employment. However, conventional tourism platforms have concentrated traffic on a few dozen 'commercialized' destinations (such as Manali, Shimla, and Goa), leading to intense environmental degradation, overcrowding, local inflation, and strain on public resources. At the same time, hundreds of culturally rich, scenic, and peaceful 'offbeat' destinations (like Mainpat, Tawang, Chitrakote, and Lambasingi) remain undiscovered by mainstream tourists. Yātrā AI was conceived to bridge this gap. By utilizing state-of-the-art Generative AI capabilities combined with local geographic rules, the application generates customized, day-by-day travel plans focused on uncovering hidden gems. It supports local economies by connecting neighborhood stay partners directly to travelers, bypassing heavy aggregator commissions.

1.2 Problem Statement

Modern travel planning is highly fragmented. A typical traveler spends hours on search engines, maps, and hotel aggregators, cross-referencing budgets and lists. Furthermore, existing travel platforms suffer from major limitations:
1. Over-commercialization: Algorithms bias searches toward high-paying commercial operators.
2. Fixed Itineraries: Standard packages lack flexibility regarding budget, origin city, and travel styles.
3. High Commission Fees: Aggregators charge 15-20% commissions from local hotels, forcing them out of digital discoverability.

Yātrā AI resolves these issues by serving as an open, commission-free, AI-optimized, and visually stunning geospatial travel planner.

1.3 Proposed System

Yātrā AI provides an interactive, responsive portal. Users select their home city, budget, travel duration, and desired comfort level. The system queries custom-curated databases and Google's Gemini LLM to construct customized itineraries. Results are presented with interactive Leaflet maps, dynamic budget charts, and target-destination packing checklists. It features a decentralized hotel onboarding module that saves agreements straight to Google Sheets and alerts administrators immediately via email.

CHAPTER 2: SYSTEM REQUIREMENT ANALYSIS

2.1 Hardware Requirements

For Development & Server Hosting:
• CPU: Dual-Core Intel/AMD Processor (2.0 GHz or higher)
• RAM: 4 GB Minimum (8 GB recommended for development environment)
• Disk Space: 500 MB free space for script execution, assets, and caching logs

For Client Access:
• Any modern smartphone, tablet, or PC with browser rendering support.

2.2 Software Requirements

• Operating System: Platform-agnostic (Windows 10+, macOS, Linux)
• Programming Environment: Python 3.10+ (Flask, Requests, python-docx)
• APIs Consumed: Google Gemini API (gemini-1.5-flash)
• Web Libraries: Leaflet.js (map overlays), Chart.js (budget division)
• Fonts & Assets: Google Fonts (Outfit, Inter), FontAwesome 6 Icons
• Domain & Hosting: Cloud hosting services (e.g. Render, GitHub Pages)

CHAPTER 3: ARCHITECTURE & DATABASE DESIGN

3.1 MVC Architectural Flow

Yātrā AI uses a Model-View-Controller (MVC) architectural design pattern implemented over Flask:
1. Model: Python data scripts (such as ai_engine.py) structure prompt schemas, interface with SQLite caches, and communicate with the Gemini API.
2. View: The frontend (index.html, style.css, app.js) renders the visual elements, maps, and charts to the user.
3. Controller: The Flask controller (app.py) handles HTTP requests, handles API queries, generates dynamically compiled sitemaps, and serves static files.

3.2 Decoupled Partner Database Pipeline

A traditional SQL database requires constant hosting fees and maintenance. To keep Yātrā AI lightweight, the B2B Stay Partnership pipeline uses a decoupled, serverless database model. When a hotel submits the partnership form, the frontend dispatches two parallel queries:
1. Email Dispatch: Uses FormSubmit's AJAX API to route verified agreement details to the admin's email inbox.
2. Live Spreadsheet Log: Sends a POST request (CORS-bypassed using 'no-cors' mode) to a custom Google Apps Script Web App. The script automatically parses the payload and appends it to a secure, permanent Google Sheet, creating a real-time collaborative database.

CHAPTER 4: ADVANCED IMPLEMENTATION DETAILS

4.1 Prompt Engineering and Gemini API Integration

The system formats travel parameters into a precise JSON structure using system instructions to prevent LLM hallucinations. The prompt forces the model to output a structured JSON schema, containing:
• destination, state, description, duration_days

• cost_summary (estimated_transit, estimated_stay, estimated_food, total_estimated)
• day_by_day list of activity guidelines, sights, and entry ticket prices
This ensures the program parses the API response directly without manual regex mapping.

4.2 Dynamic Map Tile Theme Adaptation

The Leaflet.js configuration automatically swaps tile layers based on the client's selected theme mode. When the user clicks the theme switcher in the header, the body toggles the '.light-theme' class. The application removes the active map instance and reconstructs it:
• Dark Mode: Renders CARTO DarkMatter tiles ('https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png')
• Light Mode: Renders CARTO Voyager tiles ('https://{s}.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}{r}.png')
This delivers a high-contrast map visual in both interfaces.

4.3 Stereoscopic 3D Parallax & Spotlight

To make the site look visually stunning and cute, we built a hardware-accelerated 3D parallax system. A mousemove listener tracks coordinates relative to the screen center and sets CSS properties:
• --mouse-x & --mouse-y: Position a radial gradient spotlight ('body::before') that highlights background honeycomb lines.
• --tilt-x & --tilt-y: Used by background travel floaters (🛩️, ☁️, 🎈) to translate on the Z-axis ('translate3d'). Emojis with high Z-values move faster than background layers, producing a true 3D visual depth.

4.4 Premium PDF Layout Architecture

When premium travel PDF downloads are unlocked via the paywall, the app appends the '.unlocked-premium' class to the body and triggers the native 'window.print()' window. CSS print media queries (@media print) hide navigation buttons, sidebars, and commercial affiliate links. It styles a premium, double-bordered Cover Page containing the travel duration, style, and total budget, formatting it like a travel catalog.

CHAPTER 5: TESTING & DOCUMENTATION

5.1 Search Console Verification & Crawling

To ensure the application ranks on Google searches, we integrated Search Console tools. A custom route in app.py serves the HTML verification file dynamically. We compiled and submitted a structured 'sitemap.xml' mapping all routes to Google's indexing crawlers, making Yātrā AI discoverable for organic search keywords.

5.2 Verification Table

Test Case / Module	Testing Procedure	Verification Status
Gemini AI API Prompt	Submit itinerary request with low budget	**PASS - Costs adjusted properly**
Live Google Sheet webhook	Register a stay partner with promo code	**PASS - Row logged in spreadsheet instantly**
Theme Swapping Maps	Toggle light/dark mode with active route	**PASS - Tiles swapped and redrawn dynamically**
Google Sitemap Crawling	Query index URL /sitemap.xml	**PASS - Valid XML format indexed by Google Console**

CHAPTER 6: CONCLUSION & PROJECT SUMMARY

Yātrā AI successfully demonstrates a modern travel planning interface designed to highlight offbeat destinations in India. By integrating generative AI, interactive mapping, dynamic print-media stylesheets, and a database pipeline linked to Google Sheets, the application acts as an educational and commercially viable product. The implementation demonstrates proficiency in full-stack Python development, REST APIs, responsive design layouts, and search engine optimization. Future scopes include multi-lingual audio guides and direct GPS navigation overrides.